

DQ Compass

The Supervisory Mapping — Appendix C

MBA-ESG / SG GSC Datathon 2026 — *DQ Compass: a Plug-and-Play Data Quality Control Layer for EUCs*

Working Group 1

Generated 2026-09-08

The Supervisory Mapping is the bridge between the technical control layer and a supervisor's language (BCBS 239). For every control it states **which supervisory requirement it satisfies**, **what evidence it produces**, and **the hash of the run that proves it**. It is generated entirely from the Control Catalogue — there is no separate mapping document to keep in sync.

Item	Where
Generator	mapping.py :: build_mapping() / mapping_csv()
Page	/mapping (mapping_view -> templates/mapping.html)
Export	reporting/supervisory_mapping.csv (python mapping.py or dqcompass mapping)
Catalogue hook	regulatory_ref column (canonical schema, rule_authoring/schema.py)

1 What it is and why

Appendix C asks the solution to demonstrate **governance and auditability** in supervisory terms: not just “we run N checks”, but “here, control by control, is the regulatory expectation each one covers, the evidence it leaves, and proof it was actually executed”.

- **Auto-generated from the catalogue** — add / edit / retire a rule and the mapping follows on the next page load. Nothing to maintain by hand, so it can never drift.
- **One row per ACTIVE rule** (inactive rules are skipped — they are not a live control).
- **Three sub-parts** of Appendix C: C.2 (requirement -> control -> evidence -> output), C.3 (per-rule traceability to a run), C.4 (supervisory principles demonstrated).

2 How each row is built — build_mapping()

build_mapping() reads control_catalogue.csv and, for every active rule, emits a 14-field row:

Field	Source / rule
rule_id	catalogue
requirement	the rule's regulatory_ref if set; ELSE DEFAULT_REQUIREMENT[control_type] (see 3) - so every control is mapped
control_implemented	control_name
control_type / severity / owner / kpi	catalogue
logic_definition	the generated pseudo-SQL of the rule (auditor-readable)
evidence_generated	fixed statement: "Evidence Pack per run ([output_type]): metrics, exception dataset, run_summary.json, system_trace.json, run_log.jsonl; verification.json on request"
output	the rule's output_type (Error dataset / Validation report / ...)
last_run_id / _timestamp / _status / _hash	_last_run_for_rule() - see 4

3 The supervisory requirement

Two ways a control gets a requirement:

3.1 Explicit — the regulatory_ref column

A free-text column in the catalogue's canonical schema. The authoring form suggests values (schema.REGULATORY_REF_SUGGESTIONS): BCBS 239 Principles 3-7, “SG internal Data Quality framework”, “EUC governance policy”. Set it to bind a specific rule to a specific supervisory article.

3.2 Fallback — DEFAULT_REQUIREMENT[control_type]

If regulatory_ref is blank, a per-dimension default is used, so the mapping is never empty:

Control type	Default requirement
Completeness	Data completeness (BCBS 239 - Completeness)
Validity	Accuracy & integrity - conformance to defined formats / domains (BCBS 239 - Accuracy)
Uniqueness	No duplication of records or keys (BCBS 239 - Accuracy & Integrity)
Consistency	Referential integrity across data sets (BCBS 239 - Accuracy & Integrity)
Timeliness	Data timeliness / freshness (BCBS 239 - Timeliness)
Reconciliation	Reconciliation across sources (BCBS 239 - Accuracy & Reconciliation)

4 Traceability to a run — _last_run_for_rule()

This is Appendix C.3: every control must trace to a concrete execution.

```
walk runs_ledger.jsonl BACKWARDS (most recent first)
for each run: if evidence/<run_id>/<rule_id>_result.json exists:
```

```
return { run_id, timestamp, status (from that result file), run_hash }  
if no run ever executed this rule -> "never executed"
```

So each mapping row carries the **run_id**, the rule's **status** in that run, the run **timestamp** and the run **run_hash** — a tamper-evident pointer into the Evidence Pack. On the page the run_id links straight to /runs/<id>.

5 C.4 — supervisory principles demonstrated

A fixed list (mapping.PRINCIPLES), each tied to a concrete mechanism of the solution:

Principle	How DQ Compass demonstrates it
Traceability	Every control links to its rule definition, its dataset (snapshot + SHA-256 hash) and its execution run (run_id + run_hash).
Repeatability	Controls produce identical results under identical inputs - proven by "Verify this run", which re-executes the stored config against the stored snapshots.
Auditability	An independent party can reconstruct any run from evidence// (config snapshot, dataset snapshots, processing log, system trace) and validate the outcome.
Governance	Controls are owned (owner), monitored (catalogue + coverage matrix), versioned (hash-chained changelogs) and actionable (remediation_action, alerts.json).

6 Surfaces

Surface	Contents
/mapping page	The C.2 / C.3 table (Rule Requirement Control implemented Evidence generated Output Last run) followed by the C.4 principles card. The Last-run cell links to the run's Evidence Pack.
supervisory_mapping.csv	The same rows, all 14 columns, written by python mapping.py (or dqcompass mapping) into reporting/. Download button on the page. Power BI / audit-pack ready.

7 Appendix C compliance

Requirement (Appendix C)	Where satisfied	Met
C.2 Requirement -> Control -> Evidence -> Output	build_mapping() row; /mapping table	YES
C.3 Per-control traceability to an execution	_last_run_for_rule(): run_id + status + run_hash per rule	YES
C.4 Supervisory principles demonstrated	PRINCIPLES (Traceability, Repeatability, Auditability, Governance)	YES
Mapping stays in sync with the controls	generated from the catalogue on every load	YES
Machine-readable export	supervisory_mapping.csv (14 columns)	YES

8 Worked example (current demo)

14 active rules -> 14 mapped rows. Requirement source per rule:

Rule	Requirement (source)	Last run status / hash
CMP_PASS	BCBS 239 - Principle 4 (Completeness) [regulatory_ref]	PASS - b1c8ba33ffd68f60
CMP_FAIL	Data completeness (BCBS 239 - Completeness) [default]	FAIL - b1c8ba33ffd68f60
VAL_LIST_FAIL	Accuracy & integrity - formats / domains [default]	FAIL - b1c8ba33ffd68f60
UNQ_PASS	BCBS 239 - Principle 3 (Accuracy and Integrity) [regulatory_ref]	PASS - b1c8ba33...
CON_FAIL	BCBS 239 - Principle 3 (Accuracy and Integrity) [regulatory_ref]	FAIL - b1c8ba33...
TIM_FAIL	BCBS 239 - Principle 5 (Timeliness) [regulatory_ref]	FAIL - b1c8ba33...
REC_FAIL	BCBS 239 - Principle 7 (Accuracy - reporting) [regulatory_ref]	FAIL - b1c8ba33...
...	the other 7 rules take the per-dimension default	...

- 6 rules carry an explicit `regulatory_ref`; the other 8 inherit the dimension default — every control is still mapped.
- All rows point at run `0f13237e` (run_hash `b1c8ba33ffd68f60`) — the latest execution that contains each rule's result file.
- Regenerate: `python mapping.py -> reporting/supervisory_mapping.csv` (14 rules).